AI-Based Text Summarization Platform

Abstract

This project presents a full-stack AI-Based Text Summarization Platform designed to accept long textual input or PDF documents and generate concise, meaningful summaries. The goal is to reduce reading time while preserving the key ideas of the source content. The application includes text input support, PDF upload, AI-powered summarization, keyword extraction, summary statistics, and PDF export of the generated result. The system is implemented using a frontend built with HTML, CSS, and JavaScript, and a backend built with FastAPI. For summarization, the project uses a local Hugging Face transformer model for longer documents and a hybrid extractive strategy for shorter inputs to improve output quality and runtime reliability.

Special attention was given to modular code structure, explicit validation, safe file handling, offline-capable local model loading, and reproducible setup instructions. The final system is intended to be practical, robust, and suitable for demonstration and submission.

1. Project Title

AI-Based Text Summarization Platform

2. Objective

The primary objective of this project is to build a web-based application capable of:

- accepting direct text input from the user,
- accepting PDF document uploads,
- extracting and preprocessing textual content,
- generating a concise summary using AI,
- presenting the output in a readable format,
- and supporting practical usability features such as summary formatting and export.

From a learning perspective, this project also provides hands-on exposure to full-stack application development, API design, input validation, AI model integration, and document processing.

3. Problem Statement

In academic, business, and technical environments, users often deal with large amounts of text such as articles, notes, meeting minutes, reports, and documentation. Reading all of this content manually is time-consuming and often inefficient when the user only needs the most important ideas. Traditional search can find keywords, but it does not automatically produce a coherent summary.

This project addresses that problem by building an AI-assisted summarization platform that can compress a large body of text into a shorter and more digestible form. The platform is intended to support quick understanding, efficient review, and easier knowledge extraction from long textual data.

4. Brief Requirements and How They Were Implemented

The assignment brief requested a system that accepts text or PDF input, extracts content, performs AI summarization, and displays the result in a readable format. The final implementation covers these requirements as follows:

| Brief Requirement | Implementation in This Project |

| --- | --- |

| Accept text input | A large textarea is provided in the frontend interface for direct text submission. |

| Allow PDF upload | A file upload control accepts PDF files and sends them to the backend. |

| Extract text from uploaded files | The backend uses `pypdf` to extract readable text from PDF pages. |

| Clean and preprocess text | Whitespace normalization and text cleaning are performed before summarization. |

| Generate summary using AI | A local Hugging Face summarization model is used for long content. |

| Allow control over summary length | The UI provides short, medium, and detailed summary options. |

| Format output clearly | Output can be displayed as a paragraph or as bullet points. |

| Deliver clean UI and modular code | The project uses a structured frontend and service-based backend architecture. |

In addition to the mandatory features, the project also includes:

- keyword extraction,
- summary statistics,
- a PDF export feature,
- and smoke-test validation.

5. Technology Stack

The following tools and libraries were used in the project:

Frontend

- HTML
- CSS
- JavaScript

Backend

- Python
- FastAPI
- Uvicorn

AI and NLP

- Hugging Face Transformers
- PyTorch

Document Handling

- pypdf for PDF text extraction
- reportlab for PDF report and summary export generation

Testing and Validation

- FastAPI TestClient
- custom smoke test script

6. System Architecture

The application is designed as a three-layer system:

6.1 Frontend Layer

The frontend is responsible for user interaction. It provides:

- a text input area,
- a PDF upload card,
- summary-depth controls,
- output-style controls,
- a summary output panel,
- keyword display,
- statistics display,
- and an export button.

The frontend is intentionally designed to be simple and clean so the evaluator can understand the workflow immediately.

6.2 Backend Layer

The backend handles application logic and validation. Its responsibilities include:

- serving the frontend,
- accepting summarization requests,
- validating summary options,
- validating uploaded PDF files,
- extracting text from PDFs,
- cleaning and limiting input text,
- calling the summarization service,
- computing keyword and compression statistics,
- and returning structured JSON responses.

6.3 AI Engine Layer

The summarization layer uses a hybrid approach:

- For short input, the application uses extractive summarization. This avoids forcing a small neural model to summarize already-short content, which can lead to unstable or awkward outputs.
- For longer input, the application uses a local Hugging Face transformer model stored in the project cache for offline execution.

This design improves both quality and runtime reliability.

6.4 Workflow

The overall system workflow is:

`User Input -> Validation -> PDF Extraction / Text Cleaning -> Summarization -> Output Formatting -> Statistics / Keywords -> Export`

7. Project Structure

The folder structure was organized to keep the code modular and maintainable:

```
AL_Learning/
■■■ backend/
■   ■■■ main.py
■   ■■■ config.py
■   ■■■ schemas.py
■   ■■■ services/
■       ■■■ export_service.py
■       ■■■ pdf_service.py
■       ■■■ summarizer_service.py
■       ■■■ text_utils.py
■■■ frontend/
■   ■■■ index.html
■   ■■■ styles.css
■   ■■■ app.js
■■■ models/
■■■ outputs/
■■■ scripts/
■   ■■■ preload_model.py
■   ■■■ smoke_test.py
■■■ README.md
■■■ requirements.txt
■■■ PROJECT_REPORT.md
■■■ PROJECT_REPORT.pdf
```

This structure is intentionally modular so that each part of the system is easier to understand, update, and test.

8. Implementation Details

8.1 Frontend Design

The frontend was designed as a single-page dashboard with a clear input-output split. The left side is the input workspace, and the right side is the results workspace. This allows the user to understand the flow naturally:

1. Provide input
2. Choose summary settings
3. Generate summary
4. Review summary, keywords, and statistics
5. Export the result if required

The page also uses a cleaner visual system than a default form-based layout. The goal was to make the interface feel deliberate and polished rather than like a rough prototype.

8.2 API Design

The backend exposes the following key routes:

- `GET /`
- serves the main frontend page
- `GET /health`
- returns a simple application health status
- `POST /api/summarize`
- accepts text or PDF input and returns a summary response
- `POST /api/export`
- generates a downloadable PDF for the summary

The summarization response includes:

- the generated summary,
- summary format,
- source type,
- engine used,
- extracted keywords,
- summary statistics,
- optional warning message,
- and extracted character count.

8.3 PDF Processing

PDF handling is implemented carefully in the backend service layer. The upload pipeline performs:

- extension validation,
- empty file detection,
- file size limit validation,
- PDF signature validation,
- safe PDF parsing with `pypdf`,
- page count validation,
- multi-page text extraction,
- and empty-text detection.

This is important because file handling is one of the easiest places for a project demo to fail during evaluation.

8.4 Text Cleaning

Before summarization, the text is normalized by:

- removing null characters,
- collapsing repeated whitespace,
- fixing spacing around punctuation,
- and trimming unnecessary edges.

This small preprocessing step helps both model input stability and output readability.

8.5 Summarization Strategy

The application uses two summarization paths:

Short-input hybrid path

If the input is relatively short, the application uses an extractive summarizer that ranks sentences and returns the most informative ones. This was added because small abstractive models often produce weak or unnatural summaries on already-short passages.

Long-input transformer path

For longer input, the system loads a local Hugging Face summarization model. The text is chunked into smaller sections when necessary, summarized piece by piece, and recombined. This makes the system more practical for real-world longer content.

8.6 Keyword Extraction and Statistics

To provide richer output beyond a single summary block, the project also computes:

- top keywords,
- original word count,
- summary word count,
- compression ratio,
- and estimated reading time.

These additions improve usability and make the final output more informative.

8.7 Export Functionality

The project includes a PDF export endpoint that allows users to download a formatted summary document. This was implemented using `reportlab`. Although export was listed as an optional advanced feature, including it strengthens the overall submission.

9. Robustness and Error Handling

Special care was taken in the following areas:

9.1 No fragile global dependence

The backend logic is structured into config, schema, and service modules. The main route file does not contain all logic inline.

9.2 Input validation

The application validates:

- summary size,
- output format,
- file type,

- PDF size,
- PDF signature,
- extracted text availability,
- and minimum content length.

9.3 Safer model handling

The summarization model is loaded from a local cached path. If it cannot be loaded, the system can still fall back to an extractive summarization path instead of failing completely.

9.4 Better reproducibility

The submission includes:

- pinned dependencies,
- environment configuration example,
- setup instructions,
- and a smoke test script.

9.5 Cleaner reviewer experience

The project is easier to evaluate because it includes:

- a README,
- a project report,
- and structured source code.

10. Engineering Improvements

The following implementation decisions were important for making the system more reliable and complete:

| Area | Improvement in This Project |

| --- | --- |

| Code structure | Clear modular structure with route, config, schema, and service separation |

| Setup | Config-based values and documented setup instructions |

| Error handling | Explicit validation and meaningful error responses |

| Packaging | Full project structure, report, scripts, and requirements |

| User experience | Better UI, export feature, and supporting documentation |

| Validation | Smoke tests and route-level behavior checks |

11. Testing and Validation

The project was validated using multiple checks.

11.1 Code-level validation

- backend import check
- backend compile check
- service-layer smoke validation

11.2 Functional validation

- text summarization request returns HTTP 200
- PDF summarization request returns HTTP 200
- export endpoint returns a valid PDF response
- offline model loading works from the local cache

11.3 Smoke-test script

A dedicated script was added:

`scripts/smoke_test.py`

This script programmatically verifies that:

- text summarization works,
- PDF summarization works,
- and export generation works.

During validation, the smoke test passed successfully.

12. Sample Output Behavior

During test execution, the application successfully produced:

- a transformer-based summary for longer text,
- a hybrid extractive summary for shorter PDF content,
- extracted keywords,
- summary statistics,
- and a downloadable export PDF.

This confirms that the major use cases from the brief are operational.

Appendix A. Implemented Modules and Responsibilities

To properly reflect the actual work completed, the main modules of the project are summarized below:

| File | Responsibility |

| --- | --- |

| `backend/main.py` | FastAPI application setup, route definitions, input validation, response assembly |

| `backend/config.py` | Project configuration, model path, limits, and environment-driven settings |

| `backend/schemas.py` | Structured response models for summaries and exports |

| `backend/services/pdf_service.py` | PDF validation, text extraction, size checks, and error handling |

| `backend/services/summarizer_service.py` | Hybrid summarization logic, local transformer loading, chunking, and fallback behavior |

| `backend/services/text_utils.py` | Cleaning, sentence splitting, keyword extraction, word statistics, and extractive summarization |

| `backend/services/export_service.py` | Generation of downloadable summary PDF files |

| `frontend/index.html` | UI structure for text input, PDF upload, settings, and summary display |

| `frontend/styles.css` | Full visual styling and responsive layout |

| `frontend/app.js` | Frontend interaction logic, API calls, export handling, and UI state updates |

| `scripts/smoke_test.py` | Functional smoke testing for summarize and export endpoints |

| `scripts/preload_model.py` | First-run local model download and cache preparation |

This module-level separation is important because it shows that the project is not a superficial demo, but a properly organized application.

Appendix B. Example Execution Result

One sample test case used a long educational AI paragraph as input. The backend returned the following kind of structured result:

- Source type: `text`
- Engine used: `huggingface:sshleifer/distilbart-cnn-12-6`
- Original words: `125`
- Summary words: `64`
- Compression ratio: `0.51`
- Estimated reading time: `0.32 minutes`
- Sample extracted keywords: `learning`, `tools`, `study`, `different`, `students`

Representative summary output:

Artificial intelligence is transforming education by supporting personalized learning. Schools can use AI tools to identify learning gaps more quickly and recommend targeted study plans for different students. AI tools can also help generate practice questions, summarize classroom discussions, and organize study resources for exams. At the same time, institutions must address privacy, transparency, and bias so automated systems do not create unfair outcomes.

This example demonstrates that the system is producing structured output beyond a plain text response and is returning the extra value-added fields documented in the brief and in the interface.

13. UI and User Experience Notes

The brief asked for a clean UI design. This requirement was taken seriously rather than treated as an afterthought. The interface was designed to:

- make the workflow obvious,
- reduce clutter,
- separate input and output clearly,
- and provide contextual feedback to the user.

Status messages are shown for:

- missing input,
- processing,

- success,
- fallback behavior,
- and failure.

This helps the application feel more complete and professional during a demo.

14. Limitations

Even though the project is complete and functional, there are still realistic limitations:

- CPU-based transformer inference is slower than GPU inference.
- PDF extraction works best on text-based PDFs and not scanned image-only PDFs.
- The model used is lightweight to keep the project practical in a local environment.
- The quality of generated summaries can vary with the structure of the source text.

These limitations are acceptable for the scope of the assignment and are common in many summarization systems.

15. Future Enhancements

If this project were extended further, the following improvements would be useful:

- multi-language summarization,
- OCR support for scanned PDFs,
- user authentication,
- summary history,
- topic clustering,
- chat with document,
- and advanced model selection.

These align well with the “optional advanced features” direction from the project brief.

16. Learning Outcomes

By completing this project, the following learning outcomes were achieved:

- understanding the practical use of AI summarization in a web application,
- designing a full-stack project with clear separation of concerns,
- integrating transformer-based NLP workflows,
- handling PDFs safely in backend systems,
- adding usability features beyond the minimum requirements,
- and improving software quality through better structure, validation, and usability.

17. Conclusion

The AI-Based Text Summarization Platform successfully fulfills the main requirements of the assignment and extends them with useful additional features. It supports direct text and PDF input, generates readable summaries, offers formatting controls, displays keywords and document statistics, and allows summary export as PDF.

The code is modular, the setup is reproducible, the validation is stronger, and the final package is complete enough for presentation and academic evaluation. As a result, this project is not only functional, but also aligned with what is expected from a polished academic AI application project.